

Module Programmation mathématique avancée 2

TP 1 : Débuter avec Python — Compte rendu

Importation des packages

```
import numpy as np
from scipy.optimize import linprog
import math
import matplotlib.pyplot as plt
```

- **numpy** (alias np) : bibliothèque de calcul numérique ; elle fournit le type ndarray (tableaux/matrices) et toutes les opérations vectorisées associées ;
- **scipy.optimize.linprog** : fonction de résolution de programmes linéaires (elle servira dans les TP d'optimisation suivants) ;
- **math** : fonctions mathématiques de base (sqrt, cos, exp, ...) opérant sur des scalaires ;
- **matplotlib.pyplot** (alias plt) : module de tracé de graphiques.

Dans la suite, chaque commande saisie est précédée de « >>> » (ou « ... » pour la suite d'un bloc) ; la ou les lignes suivantes, en gris, sont le résultat affiché par Python. Chaque groupe de commandes est ensuite expliqué.

1) Création et indexation d'une matrice NumPy

```
>>> A = np.array([[1,2],[3,4]])
>>> A[0,0]
1
>>> A[:,1]
array([2, 4])
>>> A[0,:]
array([1, 2])
>>> n1,nc = A.shape
>>> print(n1,nc)
2 2
```

Explication

- `np.array([[1,2],[3,4]])` crée un tableau à 2 dimensions (matrice 2×2) à partir d'une liste de listes : chaque sous-liste est une ligne ;
- `A[0,0]` renvoie l'élément situé ligne 0, colonne 0, soit 1 : **l'indexation commence à 0** en Python ;
- `A[:,1]` extrait la colonne d'indice 1 en entier (« : » signifie « toutes les lignes ») : `array([2, 4])` ;
- `A[0,:]` extrait la ligne d'indice 0 en entier : `array([1, 2])` ;

- A.shape renvoie le tuple (nombre de lignes, nombre de colonnes) = (2, 2), que l'on décompose dans les variables nl et nc ; print(nl,nc) affiche donc « 2 2 ».

2) Type, dimensions, minimum et maximum

```
>>> c = np.array([[4,-2,-2]])
>>> print(type(c))
<class 'numpy.ndarray'>
>>> print(c.ndim)
2
>>> m = np.amin(c)
>>> print(m)
-2
>>> print(np.argmin(c))
1
>>> n = np.amax(c)
>>> print(np.argmax(c))
0
```

Explication

- type(c) montre que c est un objet **numpy.ndarray** (et non une liste Python) ;
- c.ndim = 2 : c a été créé avec des doubles crochets, c'est donc un tableau à 2 dimensions de forme (1, 3) — une ligne, trois colonnes ;
- np.amin(c) renvoie la plus petite valeur du tableau : -2 ;
- np.argmin(c) renvoie l'**indice** (dans le tableau aplati) de la première occurrence du minimum : la valeur -2 apparaît d'abord en position 1 ;
- np.amax(c) renvoie la plus grande valeur (4) et np.argmax(c) l'indice de sa première occurrence : 0.

3) arange, reshape et zeros

```
>>> a = np.array([[1.2,2.5],[3.2,1.8],[1.1,4.3]])
>>> v = np.arange(0,10).reshape(2,5)
>>> print(v)
[[0 1 2 3 4]
 [5 6 7 8 9]]
>>> print(a.reshape(3,2))
[[1.2 2.5]
 [3.2 1.8]
 [1.1 4.3]]
>>> print(np.zeros(shape=(2,4)))
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

Explication

- np.arange(0,10) génère les entiers 0, 1, ..., 9 (borne 10 exclue) ; .reshape(2,5) réorganise ces 10 valeurs en 2 lignes × 5 colonnes ;

- `a.reshape(3,2)` : `a` possède déjà 3 lignes et 2 colonnes, le tableau affiché est donc identique ; `reshape` ne modifie jamais les valeurs, il ne change que la disposition, et le nombre total d'éléments doit être conservé (ici $3 \times 2 = 6$) ;
- `np.zeros(shape=(2,4))` crée une matrice 2×4 remplie de zéros **flottants** (d'où l'affichage « 0. »).

4) Ajouter, insérer et supprimer des lignes/colonnes

```
>>> b = np.array([[4.1,2.6]])
>>> c = np.append(a,b,axis=0)
>>> print(c)
[[1.2 2.5]
 [3.2 1.8]
 [1.1 4.3]
 [4.1 2.6]]
>>> d = np.array([[7.8],[6.1],[5.4]])
>>> print(np.append(a,d,axis=1))
[[1.2 2.5 7.8]
 [3.2 1.8 6.1]
 [1.1 4.3 5.4]]
>>> print(np.insert(a,1,b,axis=0))
[[1.2 2.5]
 [4.1 2.6]
 [3.2 1.8]
 [1.1 4.3]]
>>> print(np.delete(a,1,axis=0))
[[1.2 2.5]
 [1.1 4.3]]
```

Explication

- `np.append(a,b,axis=0)` ajoute la ligne `b` **en bas** de `a` (`axis=0` = le long des lignes) : on obtient une matrice 4×2 . Attention : `append` renvoie un **nouveau** tableau, `a` n'est pas modifié ;
- `np.append(a,d,axis=1)` ajoute la colonne `d` **à droite** de `a` (`axis=1` = le long des colonnes) : matrice 3×3 ; `d` doit avoir le même nombre de lignes que `a` ;
- `np.insert(a,1,b,axis=0)` insère la ligne `b` à la position d'indice 1 ; les anciennes lignes 1 et 2 sont décalées vers le bas ;
- `np.delete(a,1,axis=0)` supprime la ligne d'indice 1 (la ligne `[3.2 1.8]`) et renvoie la matrice 2×2 restante.

5) Extraction de sous-tableaux (slicing)

```
>>> v = np.array([[1.2,2.5],[3.2,1.8],[1.1,4.3]])
>>> print(v[0:2,:])
[[1.2 2.5]
 [3.2 1.8]]
>>> print(v[-1,:])
[1.1 4.3]
```

```
>>> print(v[-2:,:])
[[3.2 1.8]
 [1.1 4.3]]
>>> print(np.max(v,axis=0))
[3.2 4.3]
```

Explication

- `v[0:2,:]` sélectionne les lignes d'indices 0 à 1 (**la borne 2 est exclue**) et toutes les colonnes ;
- `v[-1,:]` : l'indice `-1` désigne la **dernière** ligne ;
- `v[-2:,:]` sélectionne les lignes depuis l'avant-dernière jusqu'à la fin, soit les deux dernières lignes ;
- `np.max(v,axis=0)` calcule le maximum **colonne par colonne** (l'axe 0 est parcouru) : `max(1.2, 3.2, 1.1) = 3.2` et `max(2.5, 1.8, 4.3) = 4.3`.

6) Parcours d'un tableau et calcul d'une somme

```
s = 0
for i in range(0,v.shape[0]):
    for j in range(0,v.shape[1]):
        print(v[i,j])
        s = s + v[i,j]

>>> print("Somme = ",s)
Somme = 14.100000000000001
```

Explication

- `v.shape[0] = 3` (nombre de lignes) et `v.shape[1] = 2` (nombre de colonnes) : la double boucle `for` parcourt donc tous les éléments, ligne par ligne ;
- à chaque passage, l'élément `v[i,j]` est affiché puis accumulé dans la variable `s` (initialisée à 0) ;
- la somme exacte vaut `1.2 + 2.5 + 3.2 + 1.8 + 1.1 + 4.3 = 14.1` ; Python affiche `14.100000000000001` à cause des **arrondis de l'arithmétique flottante** (les décimaux ne sont pas tous représentables exactement en binaire) ;
- remarque : en NumPy, `np.sum(v)` donnerait directement le même résultat sans boucle.

7) Définition d'une fonction : racine carrée

```
>>> def racine(x):
...     return math.sqrt(x)
>>> print(racine(4))
2.0
```

Explication

- le mot-clé **def** définit une fonction nommée `racine`, de paramètre `x` ; le corps indenté est exécuté à chaque appel et **return** renvoie le résultat ;

- `racine(4)` appelle `math.sqrt(4)` et renvoie 2.0 : un **flottant**, car `math.sqrt` renvoie toujours un float, même pour un carré parfait.

8) Les listes Python

```
>>> l = list()
>>> l.append(2)
>>> print(l)
```

Explication

- `list()` crée une liste vide (équivalent à `l = []`) ;
- `l.append(2)` ajoute l'élément 2 **en fin** de liste : contrairement à `np.append`, la méthode des listes modifie l'objet en place ;
- `print(l)` affiche `[2]`. Les listes Python sont dynamiques et hétérogènes, alors que les `ndarray` de NumPy ont une taille et un type fixés — d'où leur efficacité pour le calcul.

9) Nuage de points : $y = \sin(x)$

```
>>> x = np.linspace(0, 10, 30)
>>> y = np.sin(x)
>>> plt.scatter(x, y, marker='o')
```

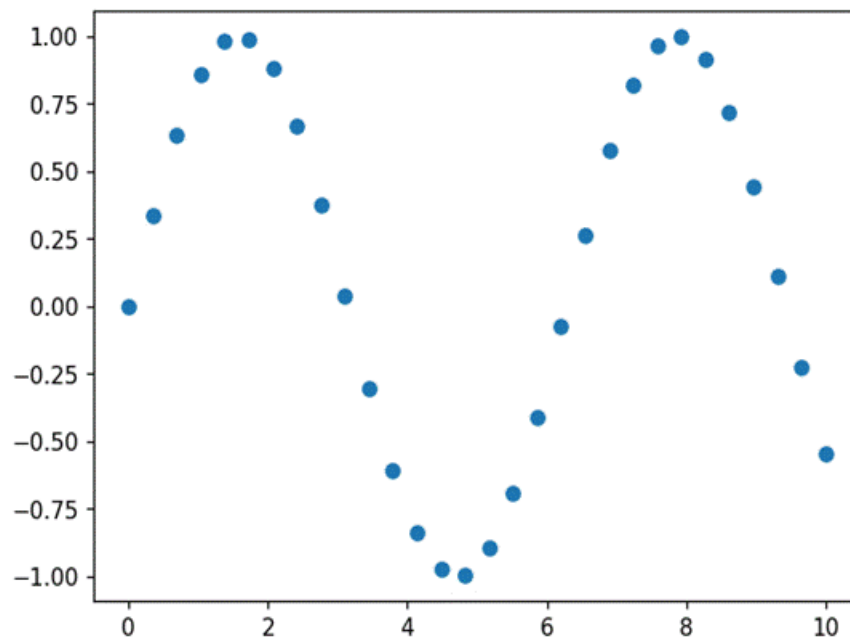


Figure 1 — Nuage de points $(x, \sin(x))$ pour 30 points équirépartis sur $[0, 10]$.

Explication

- `np.linspace(0, 10, 30)` génère 30 points **équirépartis** entre 0 et 10, bornes incluses (contrairement à `arange`, on impose ici le nombre de points) ;

- `y = np.sin(x)` applique le sinus **élément par élément** (vectorisation NumPy) : aucun besoin de boucle ;
- `plt.scatter(x, y, marker='o')` trace le nuage de points avec des marqueurs circulaires : on reconnaît la sinusoïde échantillonnée sur presque deux périodes ($10 \approx 3,2\pi$).

10) Nuage de points aléatoires (couleurs, tailles, colorbar)

```
>>> rng = np.random.RandomState(0)
>>> x = rng.randn(100)
>>> y = rng.randn(100)
>>> colors = rng.rand(100)
>>> sizes = 1000 * rng.rand(100)
>>> plt.scatter(x, y, c=colors, s=sizes, alpha=0.3, cmap='viridis')
>>> plt.colorbar()
```

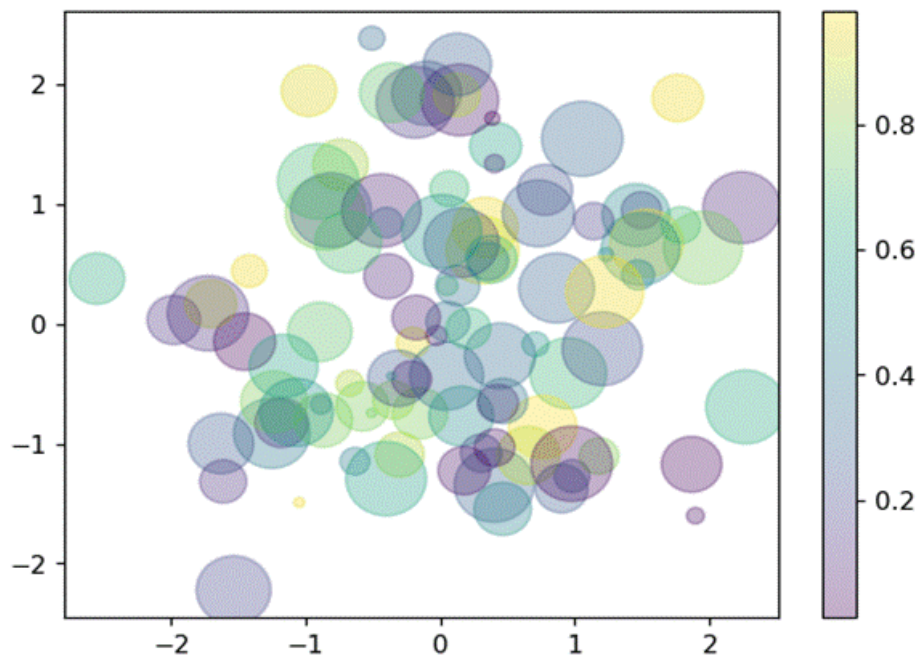


Figure 2 — 100 points gaussiens ; la couleur (colormap viridis) et la taille de chaque point sont aléatoires.

Explication

- `np.random.RandomState(0)` crée un générateur pseudo-aléatoire de **graine 0** : les tirages sont ainsi reproductibles d'une exécution à l'autre ;
- `rng.randn(100)` tire 100 valeurs selon la loi **normale** centrée réduite $N(0, 1)$ — utilisées pour les abscisses `x` et les ordonnées `y` ;
- `rng.rand(100)` tire 100 valeurs **uniformes** sur $[0, 1[$: elles servent de couleurs (`colors`) et, multipliées par 1000, de tailles (`sizes`) ;
- dans `plt.scatter`, `c=colors` associe à chaque point une couleur via la colormap 'viridis', `s=sizes` fixe la taille de chaque marqueur et `alpha=0.3` rend les points transparents pour visualiser les superpositions ;

- `plt.colorbar()` ajoute à droite l'échelle de correspondance valeur → couleur.